

Cruscotto Degenze

Diario di sviluppo

Documento	Resoconto dello sviluppo, passo per passo
Versione	1.4
Aggiornato al	18 settembre 2026
Avanzamento	6 passi completati su 9
Progetto	Casa di Cura Misericordia Navacchio S.r.l.
Specifica	Prompt_Cruscotto_Degenze.pdf, versione 1.0

A cosa serve questo documento

Questo diario racconta come è stato costruito il **Cruscotto Degenze**, passo per passo. Non è la specifica del progetto — quella è il documento *Prompt_Cruscotto_Degenze.pdf* — ma il resoconto di cosa è stato fatto davvero, perché è stato fatto così, e quali concetti nuovi sono comparsi lungo la strada.

Viene rigenerato a ogni passo completato. La data in copertina dice a quale momento corrisponde la versione che stai leggendo.

Come si rigenera

Dal terminale, nella cartella del progetto, con l'ambiente virtuale attivo:
`python docs/genera_diario.py`

Il progetto in breve

Il Cruscotto Degenze è un'applicazione web interna della Casa di Cura Misericordia Navacchio, raggiungibile solo dalla rete aziendale. Serve a vedere i pazienti ricoverati nei tre reparti ordinati per data presunta di dimissione, tenere per ognuno un'Agenda di note organizzative, seguire una checklist di dimissione di 20 voci e stampare un foglio riepilogativo.

Non è una cartella clinica. I dati clinici restano in ResMedica. Qui si registrano solo i dati minimi necessari a organizzare la degenza: nome, cognome, posto letto, date, gravità e note organizzative.

Linguaggio	Python 3.12
Applicazione web	Flask 3 con Flask-SQLAlchemy, Flask-Login, Flask-WTF
Database — Fase 1	SQLite, in un file sul PC
Database — Fase 2	Microsoft SQL Server sull'istanza sql-misnav
PDF	ReportLab, generato dal server
Server di produzione	Waitress, dietro IIS come reverse proxy HTTPS
Indirizzo finale	<code>https://degenze.cdcmisnav.local</code>

Stato di avanzamento

Passo	Contenuto	Stato
1	Impalcatura, ambiente virtuale, configurazione	FATTO
2	Database: modelli, migrazione, crea-admin	FATTO
3	Accesso: login, logout, cambio password, blocco	FATTO
4	Home «Pazienti ricoverati»	FATTO
5	Agenda delle note	FATTO

Passo	Contenuto	Stato
6	Checklist di dimissione	FATTO
7	PDF del paziente	da fare
8	Gestione utenti e registro accessi	da fare
9	Test automatici e rifinitura grafica	da fare

Decisioni prese

Le scelte concordate che si discostano dalla specifica iniziale, o che la specifica lasciava aperte. Sono annotate qui perché fra sei mesi nessuno si ricorderà il perché.

Argomento	Decisione
Chi dimette un paziente	ADMIN, MEDICO e INFERMIERE. La specifica diceva solo ADMIN: cambiato perché in reparto l'infermiere deve poter liberare il letto senza chiamare l'IT.
Messaggio di conferma	«Sei sicuro di voler eliminare il paziente? Una volta eliminato non sarà più possibile recuperare i dati. Ricordati di stampare il foglio paziente prima.»
Cancellazione dei dati	Per l'utente il paziente sparisce e non c'è modo di riportarlo indietro. Nel database la riga resta: le note devono poter risalire a chi le ha scritte, e l'amministratore di sistema può recuperare un'eliminazione sbagliata.
Utenti iniziali	Un solo account creato da terminale: administrator. Tutti gli altri vengono creati dalla pagina Gestione utenti.
Utenti di prova	Il comando flask dati-demo aggiunge quattro utenti finti, uno per ruolo. Servono perché le note di esempio devono risultare firmate da un medico e da un infermiere. È un comando facoltativo e funziona solo in sviluppo.
Password del primo admin	administrator non è obbligato a cambiare la password al primo accesso: se l'ha scelta lui stesso da terminale, il cambio forzato non protegge da nulla. L'obbligo resta per tutti gli account creati da qualcun altro.
Logout automatico	Dopo 15 minuti di inattività, controllato dal server. Nessun avviso in anticipo.
Cambio di reparto	Realizzato al passo 4 come proposto: un pulsante «Sposta» che chiede reparto e nuovo letto, con lo stesso controllo di letto occupato dell'inserimento. Note, checklist e date restano attaccate al paziente.
Nome utente	Salvato sempre in minuscolo. SQLite e SQL Server trattano le maiuscole in modo diverso: normalizzando si ottiene lo stesso risultato su entrambi.

Passo 1 — L'impalcatura

Obiettivo del passo: avere un'applicazione Flask che parte, risponde a una pagina e ha già al posto giusto tutto ciò che servirà dopo. Nessuna funzione vera, solo le fondamenta.

I file creati

Cruscotto-Degenze/	
app/	
__init__.py	create_app(): costruisce l'applicazione
config.py	configurazioni Sviluppo / Test / Produzione
estensioni.py	db, migrate, login_manager, csrf
templates/	
base.html	impalcatura HTML comune a tutte le pagine
prova.html	pagina temporanea del passo 1
static/css/	
stile.css	i colori del prototipo
tests/	
conftest.py	attrezzatura comune ai test
test_impalcatura.py	
.venv/	ambiente virtuale (escluso da Git)
.env	configurazione locale con la chiave (esclusa da Git)
.env.example	il modello vuoto, questo sì su Git
.gitignore	
requirements.txt	librerie con versione esatta
wsgi.py	punto di ingresso per Waitress
README.md	
docs/	questo diario
PromptIA/	specifica e prototipo di riferimento

I concetti nuovi

1. L'ambiente virtuale

La cartella **.venv** contiene una copia isolata di Python con soltanto le librerie di questo progetto. Serve a non farsi male: se domani un altro programma sullo stesso PC pretende una versione diversa di Flask, il Cruscotto non se ne accorge nemmeno.

L'ambiente virtuale **non va su Git**: si ricrea in un minuto a partire da requirements.txt, che elenca le librerie con la versione esatta. È così che sulla macchina virtuale finiranno le stesse identiche versioni che girano qui, senza sorprese.

2. La fabbrica create_app()

Invece di creare l'applicazione una volta sola come variabile globale, la costruiamo dentro una funzione: **create_app()**. Si chiama *application factory*, fabbrica di applicazioni, ed è il modo consigliato da Flask.

Il vantaggio è concreto. I test chiamano create_app("test") e ottengono un'applicazione con un database in memoria, che nasce e muore con il singolo test. Il comando flask run chiama create_app("sviluppo") e ottiene quella con il file SQLite. Sulla macchina virtuale sarà create_app("produzione"), con SQL Server. **Stesso codice, tre applicazioni diverse.**

3. Le estensioni in un file separato

In **app/estensioni.py** gli oggetti db, migrate, login_manager e csrf nascono vuoti, e vengono collegati all'applicazione più tardi con init_app(). Sembra un giro inutile, ma evita un problema reale.

Il futuro `models.py` avrà bisogno di **db** per definire le tabelle. Se `db` nascesse dentro `create_app()`, `models.py` dovrebbe importare l'applicazione per averlo — ma l'applicazione, a sua volta, importa `models.py`. I due file si aspetterebbero a vicenda e Python si fermerebbe con un errore. Si chiama **import circolare**, e questo è il modo standard di evitarlo.

4. Il file `.env` e i segreti

Il file `.env` contiene la `SECRET_KEY`, cioè la chiave con cui vengono firmati i cookie di sessione. Chi la conosce può fabbricarsi un cookie e farsi passare per l'amministratore senza sapere nessuna password.

Per questo il `.env` è **escluso da `.gitignore`** e non finisce mai su Git. Su Git va soltanto `.env.example`, che ha i nomi delle variabili e i commenti, ma nessun valore. Chi installa il programma altrove lo copia, lo rinomina in `.env` e ci mette i propri valori.

Da fare prima della produzione

In sviluppo, se la `SECRET_KEY` manca, il programma usa una chiave di ripiego per non bloccare il lavoro. In produzione invece si rifiuta di partire: meglio un errore chiaro all'avvio che scoprire in reparto che le sessioni sono firmate con una chiave nota.

Le intestazioni di sicurezza

Ogni risposta del server porta con sé quattro intestazioni HTTP, aggiunte in un punto solo dentro `create_app()`:

Intestazione	A cosa serve
X-Content-Type-Options	Il browser non deve indovinare il tipo di un file: se diciamo che è testo, lo tratta come testo e non come programma.
X-Frame-Options	Nessun altro sito può incorniciare il Cruscotto dentro una propria pagina per ingannare l'utente.
Referrer-Policy	L'indirizzo delle pagine interne non viene comunicato a siti esterni.
Content-Security-Policy	Consente solo risorse servite da questo stesso sito. Vale anche come rete di protezione: se per distrazione qualcuno aggiungesse un riferimento a una CDN, smetterebbe di funzionare subito qui, invece che in reparto dove non c'è internet.

Un imprevisto e come è stato risolto

Al primo giro dei test, Flask-Login si è fermato con l'errore *Missing user_loader*. Il motivo: appena viene attivato, Flask-Login pretende una funzione che sappia ricaricare l'utente dalla sessione a ogni richiesta — e quella funzione ha bisogno del modello Utente, che nascerà al passo 2.

Soluzione: `login_manager` resta dichiarato in `estensioni.py`, ma viene collegato all'applicazione al passo 3, quando avrà di che lavorare. In `app/__init__.py` c'è un commento che lo spiega, così fra un mese il motivo è ancora lì.

Come si verifica che funzioni

```
..\env\Scripts\Activate.ps1  
flask run
```

Poi si apre **http://127.0.0.1:5000**: deve comparire un riquadro bianco su sfondo grigio chiaro con scritto «Il Cruscotto funziona» nel verde del prototipo. Per fermare il server: Ctrl+C.

Se PowerShell rifiuta di eseguire Activate.ps1, va sbloccato una volta sola con: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

I test automatici si lanciano con:

```
pytest -v
```

Al passo 1 sono due: uno controlla che la pagina risponda con il testo giusto, l'altro che le quattro intestazioni di sicurezza ci siano davvero. Il secondo sembra eccessivo adesso, ma serve fra sei mesi: se qualcuno le togliesse per sbaglio, il test lo direbbe subito.

Esito

- pytest: 2 test su 2 passati.
- La pagina risponde con codice 200 e contiene il testo atteso.
- Il foglio di stile viene servito correttamente.
- git status non vede né .env né .venv: i segreti restano fuori dal controllo versione.

Passo 2 — Il database

Obiettivo del passo: le otto tabelle esistono davvero, contengono già i dati che non cambiano mai (i tre reparti e le 20 voci della checklist), e c'è un amministratore con cui poter entrare al passo 3.

I file creati

```
app/
  models.py          le otto tabelle, descritte come classi Python
  dati_fissi.py      i tre reparti e le 20 voci della checklist
  comandi.py         flask crea-admin, flask dati-demo
  formati.py         conversioni fra UTC e ora italiana
migrations/
  versions/          la prima migrazione, generata e poi completata a mano
tests/
  test_modelli.py    20 test sulle regole del database
```

Le otto tabelle

Tabella	Cosa contiene
reparto	I tre reparti. Tabella fissa, tre righe.
utente	Chi usa il programma. Gli utenti non si cancellano mai: per togliere l'accesso si usa attivo = falso.
paziente	Nome, cognome, letto, date, gravità, stato. Nessun dato clinico.
nota	Le note dell'Agenda, con la firma dell'autore copiata dentro.
nota_versione	Il testo precedente di una nota modificata.
voce_checklist	Le 20 voci, con codice fisso.
checklist_spunta	Una riga per ogni voce spuntata di ogni paziente.
evento_audit	Il registro accessi. Ci si scrive soltanto, non si modifica e non si cancella.

I concetti nuovi

1. Il modello e l'ORM

Un **modello** è una classe Python che rappresenta una tabella: ogni attributo è una colonna, ogni oggetto della classe è una riga. Lo strumento che fa da traduttore si chiama **ORM** — *Object-Relational Mapper* — e nel nostro caso è SQLAlchemy.

Concretamente: scriviamo `paziente.cognome = "Rossi"` seguito da `db.session.commit()`, e SQLAlchemy genera l'istruzione SQL UPDATE corrispondente. Il guadagno non è solo la comodità. È che **lo stesso codice funziona su SQLite e su SQL Server**: le differenze fra i due le conosce lui. Ed è impossibile sbagliare in modo pericoloso, perché non costruiamo mai un'istruzione SQL incollando pezzi di testo — il modo in cui nascono le vulnerabilità chiamate SQL injection.

Nei modelli usiamo `db.Unicode` e `db.UnicodeText` invece di `db.String` e `db.Text`. Su SQLite è identico, ma su SQL Server significa colonne `NVARCHAR` invece di `VARCHAR` — cioè lettere accentate salvate come si deve. In una colonna `VARCHAR` «Continuità» diventerebbe «Continuit?». C'è un test apposta che lo controlla.

SQLAlchemy sceglie quella giusta a seconda del database.

Accanto c'è un **vincolo CHECK** che impedisce una dimissione precedente all'arrivo. Il controllo lo farà anche l'applicazione, con un messaggio gentile in italiano; ma averlo anche nel database significa che nessun bug futuro potrà aggirarlo. Le regole importanti vanno scritte due volte.

5. La firma fotografata nella nota

La tabella nota ha tre colonne che sembrano un doppione: autore_id, autore_nome e autore_ruolo. Perché salvare nome e ruolo, se dall'id si risalirebbe all'utente?

Perché una firma deve raccontare com'erano le cose **quel giorno**. Se Carlo Bianchi passasse da infermiere a medico, le note che ha scritto da infermiere devono restare firmate «Infermiere: Carlo Bianchi». Con il solo id, cambiando il suo ruolo cambierebbero retroattivamente anche tutte le sue note vecchie.

6. UTC nel database, ora italiana a schermo

Tutti gli orari si salvano in UTC e si convertono in ora italiana solo al momento di mostrarli. Il motivo è l'ora legale: nella notte in cui le lancette tornano indietro, le 02:30 italiane esistono **due volte**. Se salvassimo l'ora locale, due note scritte a un'ora di distanza risulterebbero scritte nello stesso istante e non ci sarebbe modo di sapere quale è venuta prima. In UTC il tempo scorre sempre dritto.

Fa eccezione data_nota, che è la data *italiana* del giorno in cui la nota è stata scritta: una nota battuta all'una di notte del 17 deve comparire sotto il 17, non sotto il 16.

I comandi nuovi

Comando	Cosa fa
flask db upgrade	Porta il database all'ultima versione applicando le migrazioni che mancano. È il comando che si lancia dopo ogni aggiornamento.
flask db migrate -m "..."	Confronta i modelli con il database e scrive una nuova migrazione. Va sempre riletta prima di applicarla.
flask crea-admin	Crea il primo amministratore. Si lancia una volta sola, subito dopo l'installazione.
flask dati-demo	Carica sei pazienti, sette note e quattro utenti di prova. Solo in sviluppo. Con --azzera riparte da zero.
flask controlla-dati-fissi	Verifica che i tre reparti e le 20 voci ci siano. Utile dopo un aggiornamento sulla macchina virtuale.

Un imprevisto e come è stato risolto

Il comando flask dati-demo si rifiutava di partire dicendo di essere in produzione, pur girando in sviluppo. Il controllo leggeva la variabile DEBUG — che la configurazione imposta a vero.

Causa: il comando flask, mentre carica l'applicazione, **riscrive DEBUG a falso** se non trova la variabile d'ambiente FLASK_DEBUG. Quindi DEBUG vale vero quando l'applicazione viene avviata normalmente, ma falso dentro i comandi da terminale: non è un indicatore affidabile di dove stiamo girando.

Soluzione: una variabile nostra, **AMBIENTE**, che vale «sviluppo», «test» o «produzione» e che nessuno tocca alle nostre spalle. Lezione generale: quando una decisione importante dipende da un valore, conviene che quel valore sia nostro.

I test

Da 2 test siamo passati a **22**. Non provano Python: provano le regole che abbiamo chiesto al database di far rispettare. I più significativi:

- la password non compare mai in chiaro nell'impronta salvata;
- due utenti con la stessa password hanno impronte diverse (il sale);
- un letto occupato non si può riassegnare;
- lo stesso numero di letto in due reparti diversi è ammesso;
- il letto di un paziente eliminato torna libero;
- una dimissione precedente all'arrivo viene rifiutata;
- una dimissione nello stesso giorno dell'arrivo è ammessa;
- «Continuità» e «Setting 1 — Low Care 1» tornano dal database intatti.

I test girano su un database **in memoria**, che nasce e muore con il singolo test: non possono sporcarsi a vicenda e non toccano mai il cruscotto.db vero.

Come si verifica che funzioni

```
flask controlla-dati-fissi
pytest -v
```

Il primo comando deve rispondere «Dati fissi completi», il secondo deve dare 22 test passati.

Esito

- Otto tabelle create, con due indici e un vincolo CHECK sul paziente.
- Tre reparti e 20 voci della checklist inseriti dalla migrazione.
- Amministratore administrator creato, che entra con la sua password.
- Dati di prova caricati: 6 pazienti, 7 note, 7 spunte, 4 utenti finti.
- pytest: 22 test su 22 passati.

Passo 3 — L'accesso

Obiettivo del passo: entrare davvero nel Cruscotto. Pagina di accesso con l'aspetto del prototipo, uscita, cambio password obbligatorio, blocco dopo cinque tentativi falliti, disconnessione per inattività e pagina «Il mio profilo».

I file creati

app/	
permessi.py	chi può fare cosa: la matrice dei ruoli
audit.py	registra_evento(): il registro accessi
sessione.py	scadenza per inattività e obbligo cambio password
auth/	
__init__.py	il blueprint dell'area accesso
form.py	i form, con il token CSRF
servizi.py	le regole: tentativi, blocco, requisiti password
routes.py	le pagine: accedi, esci, cambia-password, profilo
pazienti/	
__init__.py	blueprint dell'area pazienti
routes.py	home segnaposto (diventa vera al passo 4)
templates/	
base.html	impalcatura minima
base_app.html	impalcatura con la barra laterale
_messaggi.html	i messaggi temporanei
auth/	login, cambio password, profilo
pazienti/	home
errori/	403, 404, 500 in italiano
static/css/	
stile.css	il CSS completo del prototipo
tests/	
test_accesso.py	36 test sull'accesso e sui permessi

I concetti nuovi

1. Il blueprint

Un **blueprint** è un pezzo di applicazione con le sue pagine, che poi viene innestato nell'applicazione vera. Serve a non ritrovarsi, fra sei mesi, con un unico file da duemila righe in cui la funzione del login sta accanto a quella che genera il PDF.

Il Cruscotto ne avrà cinque: auth, pazienti, agenda, pdf, utenti. I primi due esistono già. Dentro ogni blueprint la divisione si ripete: **routes.py** si occupa delle pagine, **servizi.py** delle regole. La regola «dopo 5 tentativi falliti l'account si blocca» non ha niente a che vedere con l'HTTP, e infatti i test che la verificano non aprono nessuna pagina.

2. La sessione

Come fa il server a ricordarsi chi sei fra una pagina e l'altra? Con un **cookie di sessione**: un pezzetto di dati che il browser riporta indietro a ogni richiesta. Dentro c'è soltanto l'id dell'utente, non l'utente intero.

Il cookie è **firmato** con la SECRET_KEY. Non è cifrato — chi lo apre vede che contiene l'id 3 — ma non può cambiarlo in 1 per diventare amministratore: la firma non tornerebbe. Ecco perché la SECRET_KEY è il segreto più importante del programma, e perché cambiarla costringe tutti a rifare l'accesso.

Il cookie ha tre protezioni: **HttpOnly** lo rende invisibile al JavaScript della pagina, così uno script malevolo non può rubarlo; **SameSite=Lax** impedisce che venga allegato a richieste partite da altri siti; **Secure**, attivo solo in produzione, vieta di farlo viaggiare senza HTTPS.

3. Il token CSRF

Un esempio concreto. Sei collegato al Cruscotto. Apri in un'altra scheda una pagina qualsiasi, che di nascosto invia al Cruscotto la richiesta «elimina il paziente 12». Il tuo browser allega da solo il cookie di sessione, e per il server la richiesta sembra tua.

Il **token CSRF** impedisce questo: ogni form contiene un valore casuale che il server ha generato per quella sessione, e le richieste che non lo riportano vengono rifiutate con un errore 400. La pagina esterna quel valore non può conoscerlo, perché non può leggere il contenuto delle nostre pagine.

Nei template lo inserisce `form.hidden_tag()`. È l'unica riga che non va mai dimenticata quando si scrive un form nuovo: se manca, il form semplicemente non funziona, il che per fortuna se ne accorge subito.

Le regole dell'accesso

Il messaggio di errore è sempre lo stesso

Utente inesistente, password sbagliata, account disattivato, account bloccato: in tutti e quattro i casi la risposta è «Nome utente o password non corretti». Se il messaggio cambiasse a seconda del motivo, chi prova nomi a caso scoprirebbe quali account esistono davvero, che è metà del lavoro.

Un dettaglio meno ovvio: anche il tempo parla

Se il nome utente non esiste, il programma calcola comunque un'impronta della password e la butta via. Sembra uno spreco, ed è voluto: senza, la risposta per un utente inesistente arriverebbe molto più in fretta di quella per uno esistente, e basterebbe cronometrare le risposte per capire quali nomi sono buoni. Le due strade devono durare lo stesso.

Il blocco dopo cinque tentativi

Ogni password sbagliata incrementa un contatore. Al quinto tentativo l'account resta bloccato per 15 minuti, e in quel periodo **rifiuta anche la password giusta**. Il contatore viene azzerato insieme al blocco, così passati i 15 minuti l'utente ha di nuovo cinque tentativi e non uno solo.

Un accesso riuscito azzerava tutto. Sono cinque tentativi falliti *consecutivi*, non cinque in assoluto.

Poiché il messaggio resta generico, un utente bloccato non capirebbe cosa gli sta succedendo. Per questo sotto il form c'è una riga fissa che spiega la regola: «Dopo 5 tentativi falliti l'accesso resta bloccato per 15 minuti». Informa chi ha diritto di saperlo senza rivelare nulla a nessun altro.

Il cambio password obbligatorio

Chi ha una password temporanea non può aprire nessun'altra pagina: viene sempre rimandato al cambio password. Il motivo non è formale: la password temporanea l'ha scelta l'amministratore, quindi finché resta quella l'amministratore può entrare al posto suo — e le note risulterebbero scritte dall'utente.

Fanno eccezione la pagina del cambio password stessa e l'uscita. Senza la seconda, chi entra con l'account sbagliato resterebbe intrappolato.

Requisiti della nuova password: almeno 8 caratteri, almeno una lettera e almeno un numero, diversa da quella attuale. Sono volutamente modesti: in reparto le password si digitano in fretta, e una regola troppo severa finisce scritta su un foglietto attaccato al monitor.

La disconnessione per inattività

Dopo 15 minuti senza toccare il programma l'utente viene disconnesso, con il messaggio «Sessione scaduta per inattività». Il valore sta nel .env.

Sono 15 minuti di **inattività**, non di collegamento: ogni richiesta fa ripartire il conto. Il controllo è scritto in un `before_request`, cioè una funzione che Flask esegue prima di ogni richiesta: così vale su tutte le pagine, comprese quelle che verranno scritte fra sei mesi da chi si sarà dimenticato di questa regola.

La matrice dei permessi

Tutti i permessi stanno in un unico file, `app/permessi.py`, come dizionario ruolo → permessi. Per rispondere a «un OSS può scaricare il PDF?» si guarda una tabella sola, invece di cercare i controlli sparsi per venti file.

Azione	Chi può
Vedere home, Agenda, checklist	tutti i ruoli
Scrivere note	ADMIN, MEDICO, INFERMIERE
Modificare o eliminare una nota	solo l'autore, nemmeno l'ADMIN
Spuntare la checklist	ADMIN, MEDICO, INFERMIERE
Cambiare la gravità	ADMIN, MEDICO, INFERMIERE
Inserire un paziente	ADMIN, MEDICO, INFERMIERE
Modificare la dimissione presunta	ADMIN, MEDICO, INFERMIERE
Dimettere un paziente	ADMIN, MEDICO, INFERMIERE
Spostare un paziente di reparto	ADMIN, MEDICO, INFERMIERE
Scaricare il PDF	tutti tranne OSS
Gestione utenti e registro	solo ADMIN
Cambiare la propria password	tutti

La regola che non si negozia

Ogni permesso va verificato sul server. Nascondere un pulsante serve a non confondere l'utente, non a impedire l'azione: chi conosce l'indirizzo può sempre provare a chiamarlo a mano scrivendolo nella barra del browser. Per questo ogni pagina protetta porta il decoratore `@richiede_permesso`, che risponde 403 prima ancora di entrare nella funzione.

Il registro accessi

Da questo passo il programma scrive nella tabella `evento_audit`: `LOGIN_OK`, `LOGIN_FALLITO`, `LOGOUT`, `PASSWORD_CAMBIATA`. Di ogni evento restano data e ora, utente, azione e indirizzo IP del PC.

La funzione `registra_evento()` **non fa il commit**: aggiunge soltanto l'evento alla sessione, e il salvataggio avviene insieme all'operazione che l'evento descrive. Così, se l'operazione fallisce a metà, non resta nel registro la traccia di una cosa che non è mai successa.

Nel registro non finisce mai il testo di una nota, né una password. Serve a sapere che alle 14:32 Carlo Bianchi ha modificato la nota 87, non a raccontare una seconda volta cosa c'era scritto. C'è un test che lo

verifica.

Le pagine di errore

403, 404 e 500 hanno ora una pagina in italiano, senza dettagli tecnici. Il motivo non è solo di garbo: mostrare la struttura interna del programma — nomi di file, righe di codice, versioni delle librerie — aiuta soltanto chi volesse attaccarlo. Il messaggio completo finisce nel log del server, dove lo legge l'IT.

La pagina 500 fa anche una cosa meno visibile: annulla la sessione del database. Se l'errore è avvenuto a metà di un salvataggio, restano modifiche in sospeso che non devono finire salvate per sbaglio alla richiesta successiva.

I test

Da 22 a 61. I nuovi riguardano l'accesso, ed è la parte dove un errore non si vede a occhio ma fa finire una nota firmata dalla persona sbagliata. I più significativi:

- utente inesistente e password sbagliata danno lo stesso messaggio;
- un utente disattivato non entra;
- cinque tentativi falliti bloccano l'account, quattro no;
- l'account bloccato rifiuta anche la password giusta;
- scaduto il blocco, l'accesso torna possibile;
- chi deve cambiare password non apre altre pagine, ma può uscire;
- una password senza numeri, senza lettere o troppo corta viene rifiutata;
- la sessione scade dopo i minuti previsti e ogni richiesta fa ripartire il conto;
- nel registro non compare mai la password digitata;
- la voce «Gestione utenti» non compare a un infermiere.

Come si verifica che funzioni

```
flask run
```

Si apre `http://127.0.0.1:5000` e compare la pagina di accesso con la card centrata del prototipo. Si entra con **administrator** / **administrator** e si arriva alla home, con nome e ruolo nella barra a sinistra.

Vale la pena provare anche:

- una password sbagliata cinque volte di fila, poi quella giusta: viene rifiutata comunque;
- l'accesso come `cbianchi` / `prova2026`: la voce «Gestione utenti» sparisce;
- scrivere `/profilo` o `/cambia-password` nella barra senza essere collegati: si viene rimandati all'accesso.

Esito

- Accesso, uscita, cambio password e profilo funzionanti.

- Blocco dopo 5 tentativi e scadenza per inattività verificati.
- Il form senza token CSRF viene rifiutato con 400.
- Registro accessi popolato: LOGIN_OK, LOGIN_FALLITO, LOGOUT, PASSWORD_CAMBIATA.
- pytest: 61 test su 61 passati.

Intermezzo — Il logo della struttura

Fra il passo 3 e il passo 4 è stato inserito il logo della Casa di Cura: completo nella pagina di accesso, ridotto al solo stemma nella barra laterale. Un lavoro da mezz'ora che ha tirato fuori due cose che vale la pena sapere.

Dove è stato messo

Pagina	Cosa compare
Accesso	Marchio completo (stemma e nome), 190 px, centrato sopra il titolo. È la pagina d'ingresso: ci sta il marchio per esteso.
Barra laterale	Solo lo stemma, 52 px, sopra la scritta «Cruscotto Degenze». La barra è larga 236 px: con il logo completo il titolo sarebbe andato a capo.

Il nome della struttura non viene scritto due volte: nella pagina di accesso lo dice il logo, e il sottotitolo è diventato «Accesso riservato al personale». Nella barra laterale lo stemma ha alt vuoto, perché il nome è già scritto accanto: un lettore di schermo che leggesse anche l'immagine ripeterebbe la stessa cosa due volte.

Prima sorpresa: il file era per la stampa, non per lo schermo

Il logo consegnato dalla grafica è un JPG in **modo CMYK**, con dentro un profilo colore ISO Coated v2: cioè descritto in inchiostri da tipografia, non in luce da monitor.

Aprendolo senza tenerne conto, il ciano del marchio risultava **#00FFFF**, un ciano elettrico da schermo. Convertendolo davvero in sRGB diventa **#00A3D4**, un azzurro più pieno e più scuro. La seconda è quella giusta: la prima erano valori di inchiostro letti come se fossero luce.

La differenza pratica

Il colore ingenuo **#00FFFF** stonava accanto al verde del Cruscotto (**#0E5A55**). Quello vero, **#00A3D4**, ci sta bene. Il colore giusto non è solo più corretto: era anche più bello, il che capita più spesso di quanto sembri.

Seconda sorpresa: 1,3 MB per un'immagine da 380 pixel

Il primo tentativo di ridimensionamento ha prodotto un PNG da 380x291 pixel che pesava **1350 KB**. Un'anomalia evidente: quell'immagine, salvata senza alcuna compressione, occuperebbe 332 KB. Doveva esserci dell'altro dentro al file.

C'era: il profilo colore ICC dell'originale, **1,8 MB**, che Pillow copiava fedelmente dentro il PNG. Pesava cinque volte l'immagine. Tanto che Pillow si rifiutava perfino di riaprire il file che aveva appena scritto.

La soluzione è in due mosse: prima **convertire** i colori in sRGB usando il profilo, poi **buttarlo via**. Cancellarlo e basta avrebbe spostato le tinte, perché i colori erano espressi secondo quel profilo. In più i colori vengono ridotti a una tavolozza di 64: il logo usa poche tinte piatte, e il JPG di partenza aveva sparso attorno a ogni contorno migliaia di sfumature quasi identiche, che il PNG comprime malissimo.

originale	2,1 MB	JPG CMYK, profilo ICC da 1,8 MB
primo tentativo	1350 KB	profilo copiato dentro il PNG
risultato	16 KB	sRGB, senza profilo, 64 colori

Il tutto sta in **docs/prepara_logo.py**, che si rilancia se un domani l'ufficio comunicazione consegnasse un logo nuovo. Due test di guardia verificano che i loghi vengano serviti e che restino sotto i 100 KB: se qualcuno copiasse l'originale dentro static senza prepararlo, se ne accorgerebbe subito.

Lezione generale: un file consegnato da altri va sempre guardato dentro prima di metterlo in produzione. «È solo un'immagine» non è una descrizione sufficiente.

Passo 4 — La home «Pazienti ricoverati»

Obiettivo del passo: il cruscotto vero. Tre riquadri, uno per reparto, con l'elenco dei ricoverati ordinato per data presunta di dimissione, la gravità cliccabile, l'inserimento di un paziente, la modifica della dimissione, lo spostamento di reparto e la dimissione.

I concetti nuovi

1. L'ordinamento naturale

In ordine alfabetico **A-10 viene prima di A-2**, perché il carattere «1» viene prima del «2». In reparto è il contrario: il letto 2 viene prima del 10.

La soluzione è spezzare la sigla nei suoi pezzi e trattare i numeri come numeri: «A-2» diventa ["a-", 2, ""] e «A-10» diventa ["a-", 10, ""]. A quel punto 2 è minore di 10 e l'ordine torna quello giusto.

Perché l'ordinamento non lo fa il database

SQL non sa ordinare in modo naturale, e il poco che sa fare si scrive in modi diversi su SQLite e su SQL Server: il codice smetterebbe di essere lo stesso nelle due fasi. I pazienti vengono quindi letti dal database e ordinati in Python. Con un centinaio di ricoverati la differenza di velocità non è misurabile.

L'ordine completo è a tre livelli: prima chi non ha una data di dimissione va in fondo e mostra «da definire»; poi si ordina per data, dalla più vicina; a parità di data, per letto in ordine naturale.

2. Il salvataggio immediato

Il pallino della gravità ruota a ogni clic — non valutato, verde, giallo, rosso, e si ricomincia — e si salva subito, senza ricaricare la pagina. Il browser manda una richiesta **fetch** al server, riceve il nuovo stato e aggiorna il solo pallino.

Ricaricare l'intera home per cambiare un pallino sarebbe uno spreco, e in reparto si perderebbe la posizione nella pagina: con tre reparti pieni si tornerebbe ogni volta in cima.

Due accortezze meno ovvie. Il pulsante si blocca durante l'attesa: due clic rapidi farebbero avanzare il ciclo due volte e l'utente vedrebbe un colore che non ha scelto. E se il salvataggio fallisce compare un avviso che invita a ricaricare: peggio di un errore è un errore silenzioso, con l'infermiere convinto di aver segnato un paziente critico.

3. Niente JavaScript dentro l'HTML

Nel prototipo i pulsanti portavano scritto `onclick="..."` direttamente nell'HTML. Qui non è possibile: la Content-Security-Policy che abbiamo messo al passo 1 consente solo codice servito da noi, e un onclick è codice dentro la pagina. Il browser lo bloccherebbe.

Tutti i comportamenti sono quindi agganciati in **app.js**, leggendo gli attributi `data-*`. C'è anche un unico ascoltatore per tutta la pagina invece di uno per pulsante: si chiama *delega*, il clic risale fino al documento e lì si guarda da quale pulsante è partito.

Il guaio che questo evita

Un onclick bloccato dalla CSP non dà nessun errore visibile: compare solo nella console del browser. In reparto si vedrebbe soltanto un pulsante che non fa niente. C'è un test apposta che controlla che nella home non compaia nessun onclick, onchange o onsubmit.

Lo spostamento di reparto

Realizzato come proposto: un pulsante **Sposta** che apre una finestrella con reparto di destinazione e nuovo posto letto, e riusa lo stesso controllo di letto occupato dell'inserimento.

La cosa importante è che **non è un elimina e reinserisci**: la riga del paziente resta la stessa, cambia solo dove si trova. Note, checklist, date e storia restano attaccate a lui. C'è un test che lo verifica confrontando l'identificativo prima e dopo.

La dimissione del paziente

La finestra di conferma riporta il testo concordato, comprese le parole sul foglio da stampare prima. L'eliminazione è logica: lo stato passa a ELIMINATO con chi e quando, il paziente sparisce dal cruscotto e **il letto torna libero** — proprio perché l'indice univoco vale solo sui ricoverati.

Un dettaglio: le route rifiutano con 404 le operazioni su un paziente già dimesso. Serve a chi tiene aperta una pagina vecchia e clicca su un pulsante che nel frattempo non ha più senso — succede spesso, con i PC di reparto sempre accesi.

Passi 5 e 6 — Agenda e checklist

I due passi stanno insieme perché vivono nella stessa finestra: la checklist in alto, poi il campo per la nuova nota, poi l'elenco delle note.

Il pezzo di pagina che arriva dal server

L'Agenda si apre come finestra sopra la home e il suo contenuto viene chiesto al server al momento del clic. Il server non restituisce una pagina intera né dei dati grezzi: restituisce **l'HTML già disegnato**, che il JavaScript infila dentro la finestra.

Perché l'HTML lo fa il server e non il browser

Il testo delle note lo scrivono le persone. Jinja lo mette in pagina con l'escape automatico, quindi una nota che contenesse per errore o per malizia del codice resta testo. Costruendo l'HTML a mano nel browser basterebbe una dimenticanza per trasformare una nota in codice eseguibile, e il primo a farne le spese sarebbe il collega che la apre.

Insieme all'HTML viaggia anche il numero di note visibili: serve ad aggiornare il contatore sul pulsante «Agenda» nella home, che sta fuori dalla finestra e quindi non verrebbe ridisegnato.

Solo l'autore, nemmeno l'amministratore

I pulsanti Modifica ed Elimina compaiono soltanto sulle proprie note, e il server rifiuta comunque chiunque altro — **compreso l'ADMIN**.

Non è una svista della specifica: una nota è una dichiarazione firmata da una persona. Se un amministratore potesse riscriverla, la firma non varrebbe più niente, e in una struttura sanitaria è esattamente il contrario di quello che serve.

Quando una nota viene modificata, il testo precedente finisce in **nota_versione** prima di essere sovrascritto, e la nota riceve la data di modifica. «La nota diceva un'altra cosa» non deve restare una questione di parola

contro parola. L'eliminazione è logica come sempre: la nota sparisce da Agenda e PDF, ma resta nel database con chi l'ha eliminata e quando.

La checklist

Venti voci su due colonne, contatore e barra di avanzamento, contatore che diventa verde a 20/20. Ogni spunta si salva subito, senza ricaricare.

Sulla checklist **non si registra chi ha spuntato né quando**: lo esclude la specifica, ed è una scelta sensata. È uno strumento di lavoro condiviso del reparto, non un registro di responsabilità individuali. C'è un test che controlla che la tabella abbia due sole colonne, perché è il tipo di cosa che qualcuno prima o poi «migliorerebbe» aggiungendo un campo.

Se il salvataggio di una spunta fallisce, il segno viene **rimesso com'era**. Lasciarlo dove l'utente l'ha messo sarebbe peggio che non averlo messo affatto: si crederebbe di aver salvato qualcosa che non c'è.

Lo stato aperto o chiuso del riquadro viene ricordato in sessionStorage, che dura quanto la scheda del browser: la preferenza segue il turno di chi sta lavorando e non resta appiccicata al PC condiviso. Se il browser vieta sessionStorage — succede in certe configurazioni — la checklist resta semplicemente aperta: è una comodità, non deve far fallire nulla.

I test

Da 66 a **138**. I più significativi dei tre passi:

- «A-2» viene prima di «A-10», e «BOX» dopo entrambi;
- chi non ha data di dimissione va in fondo;
- il letto occupato viene rifiutato dicendo da chi;
- il letto di un paziente dimesso è riassegnabile;
- lo spostamento conserva l'identificativo e le date del paziente;
- la firma della nota non cambia se l'utente cambia ruolo;
- nemmeno l'amministratore può modificare la nota di un altro;
- la modifica salva il testo precedente in nota_versione;
- riscrivere lo stesso testo non crea una versione inutile;
- l'OSS riceve 403 su gravità, note, checklist ed eliminazione;
- nel registro non finisce mai il testo di una nota;
- nella home non compare nessun onclick.

Come si verifica che funzioni

Si entra con administrator e si guarda la home: tre riquadri, i pazienti ordinati per dimissione, i pallini colorati. Poi vale la pena provare:

- cliccare un pallino più volte: cambia colore e resta cambiato anche ricaricando;
- cliccare una data di dimissione e spostarla: l'elenco si riordina;

- aprire l'Agenda di Rossi Mario: cinque note raggruppate per giorno, una con «modificata il»;
- entrare come cbianchi / prova2026: può modificare solo le proprie note;
- entrare come gverdi / prova2026 (OSS): vede tutto ma non può toccare niente.

Prossimo passo

Passo 7 — Il PDF del paziente. Il foglio da stampare prima di dimettere qualcuno: dati del paziente, checklist su due colonne e storico delle note, generato con ReportLab. È la stessa libreria di questo diario, quindi la tecnica è già stata vista.

Poi il **passo 8** con la gestione utenti e il registro accessi, e il **passo 9** con la rifinitura grafica.